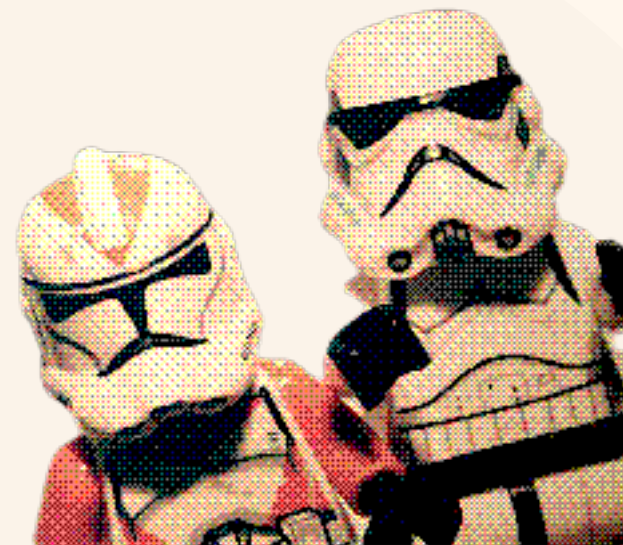


Fetch & async JavaScript

💡 Dagens fokus

- Forstå Fetch API'et og asynkron JavaScript
- Brug Git branching i praksis
- Arbejd parvist sammen om at udvikle et lille kortspil ved at bruge et eksisterende API



Dagens program

- | | | | |
|-------|----|---|--|
| 8:15 | 5 | 🎓 | Intro & agenda |
| 8:20 | 20 | 🎓 | <u>Fetch, promises, .then()</u> |
| 8:40 | 10 | 🤝 | <u>Inspicér et fetch promise i DevTools</u> |
| 8:50 | 5 | 🎓 | <u>Hvorfor er fetch kald asynkrone? Hvad hvis de var synkrone?</u> |
| 8:55 | 90 | 🤝 | <u>Byg en løsning med Deck of Cards API</u> |
| 10:25 | 10 | 🕒 | Pause |
| 10:35 | 10 | 🤝 | <u>Inspicér jeres kortspil i DevTools</u> |
| 10:45 | 20 | 👥 | <u>Vis jeres løsning ved bordet</u> |
| 11:05 | 20 | 💡 | <u>Præsenter eksempler og læring for klassen</u> |
| 11:25 | 5 | 🎓 | Tak for i dag |

1. Fetch, promises, .then()

🌐 Hvad er Fetch API?

Fetch API'et er browserens måde at hente data fra eksterne servere:

- Indbygget i alle moderne browsere
- Returnerer altid et Promise-objekt
- Bruges til at kommunikere med API'er (o.a.) over HTTP

💡 Promises – løfter om fremtidige data

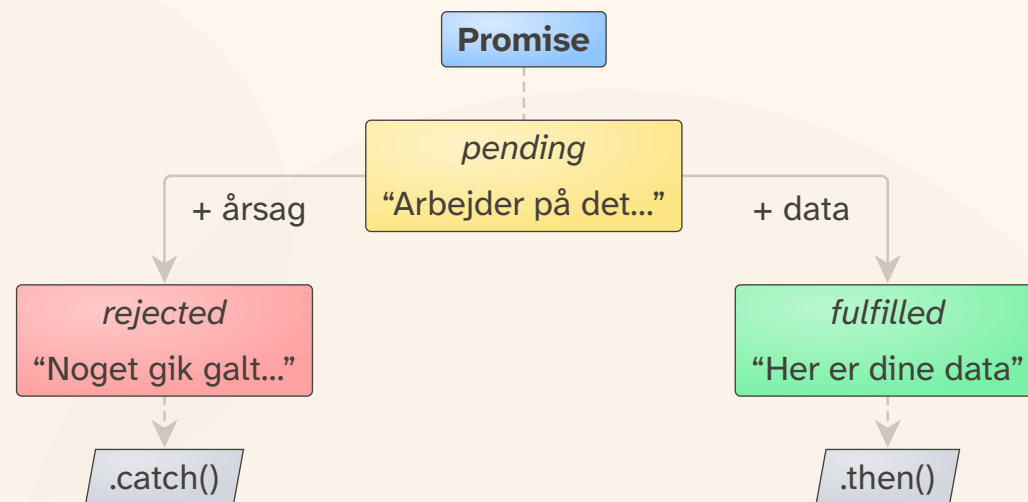
Et **Promise** er et løfte om data, der kommer senere.

Det kan befinde sig i en af tre tilstande:

pending “Jeg arbejder på det...”

fulfilled “Her er dine data”

rejected “Noget gik galt...”



1.1 .then() – “når du er klar...”

Dig

“Hey browser, kan du hente nogle data fra dette API?”

Browser

“Selvfølgelig! Her er et promise. Jeg starter arbejdet nu...”

Dig

“Okay, `.then()` når du er færdig, vil jeg gerne parse det som JSON”

Browser

“Noteret! Jeg sætter det i køen...”

Dig

“Og `.then()` når det er gjort, vil jeg logge dataene”

Browser

“Got it! Jeg husker hele kæden...”

 Eller sagt med kode:

```
fetch('https://api.example.com/data')  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error('Fejl:', error))
```

1.2 Fejlhåndtering med .catch()

Hvad sker der når noget går galt?

🔗 **Fang fejl hvor som helst i kæden**

```
fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    return response.json();
  })
  .then(data => handleSuccess(data))
  .catch(error => handleError(error));
```

Husk: `.catch()` fanger fejl fra hele kæden!

💬 **Diskutér med din sidemakker:**

I hvor mange og hvilke linjer i koden til venstre kan der opstå fejl, som ender i `catch`-blokken?

💡 **Almindelige fejlscenarier**

- Netværksforbindelsen er nede
- Serveren returnerer fejlstatus (404, 500, etc.)
- JSON parsing fejler
- Typos i URL eller kode

2. Inspicér et fetch promise i DevTools

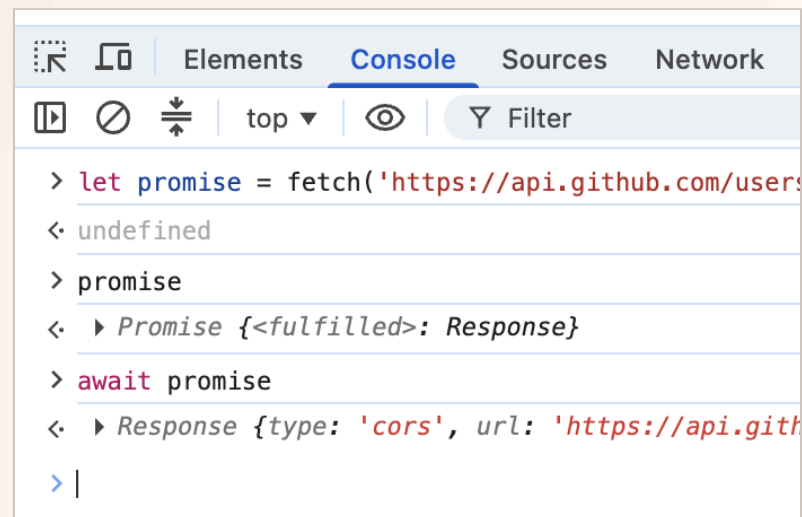
[Parvis](#) ⌚ 10 min

🔧 Afprøv i konsollen

Prøv disse linjer én ad gangen i browserens DevTools console
(Tryk Enter efter hver):

```
let promise = fetch('https://api.github.com/users/octocat');  
  
promise;  
  
await promise;  
  
promise.then(response => console.log('Response:', response));  
  
let res = await promise;  
  
let json = res.json();  
  
json;  
  
await json;
```

💬 **Hvad observerer I?** Forklar forskellene for hinanden.



3. Hvorfor er fetch kald asynkrone? Hvad hvis de var synkrone?

🕒 Synkron kode — alt stopper op

Forestil dig hvis fetch-kald var synkrone:

```
console.log("Starter...");  
let data = fetchSync('https://slow-api.com/data');  
console.log("Færdig med API-kald");
```

Problem: Browseren fryser i 3 sekunder — ingen klik, scroll eller animation virker!

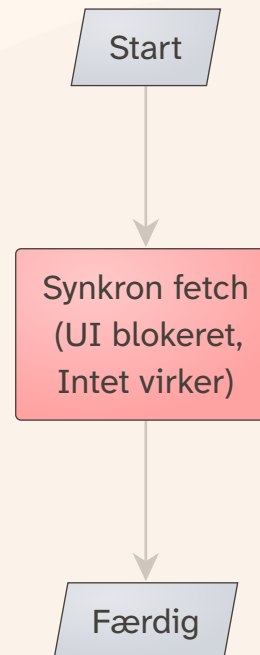
💡 Asynkron kode — alt kører videre

Med asynkron fetch kan browseren gøre andre ting mens den venter:

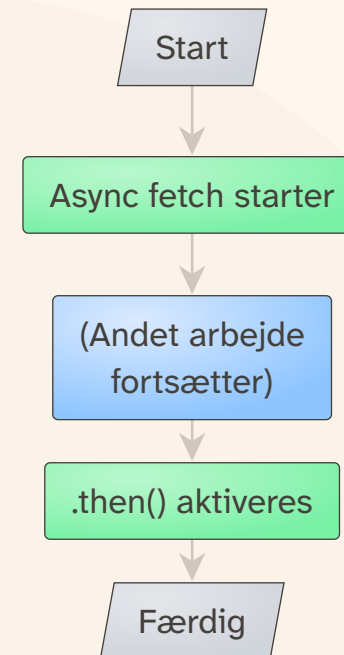
```
console.log("Starter...");  
fetch('https://slow-api.com/data')  
  .then(data => console.log("Data modtaget!"));  
console.log("Dette kører med det samme!");
```

Fordel: Browseren forbliver responsiv — brugeren kan stadig interagere med siden

Synkron:



Asynkron:



4. Byg en løsning med Deck of Cards API

♥ Parvis ⌚ 90 min

I sættes sammen i par og skal udvikle en lille kortspils-app sammen op imod et eksisterende API.

I vælger helt selv hvad spillet skal gå ud på, og hvor sofistikeret, det skal være.

🚩 The rules of Fight Club:

- 01 I skal **pair-programme** løsningen: én person koder, den anden hjælper med at tænke højt og fange fejl. Skift undervejs.
- 02 **Ingen brug af AI.** Slå Copilot fra (VS Code: **Settings > Disable AI**¹), og lad være med at bruge ChatGPT, Claude etc. Slå op i online dokumentation som rigtige udviklere ;)
- 03 I må **kun** bruge **to opslagsværker**:
 - [Deck of Cards API dokumentation](#)² (API'et I skal bruge)
 - [DevDocs.io](#) (samler dokumentation fra bl.a. MDN, så I kan søge i JS, DOM, Fetch API, etc.)
- 04 Fokus på at **lære undervejs** — snak løsningsmulighederne godt igennem. I skal kunne præsentere jeres løsning og læringer for resten af klassen senere.

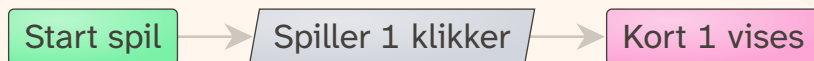
¹<vscode://settings/chat.disableAIFeatures>

²<https://deckofcardsapi.com>



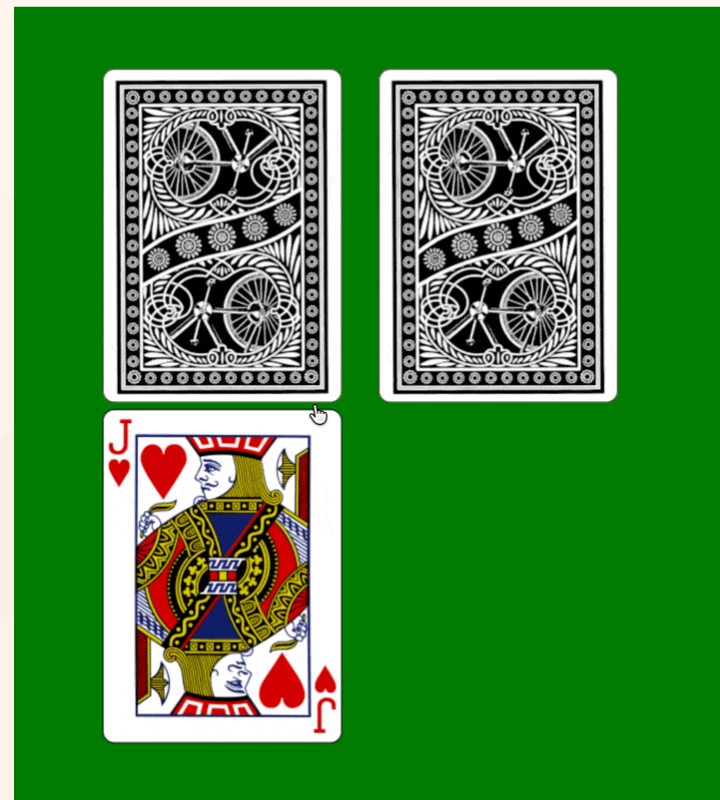
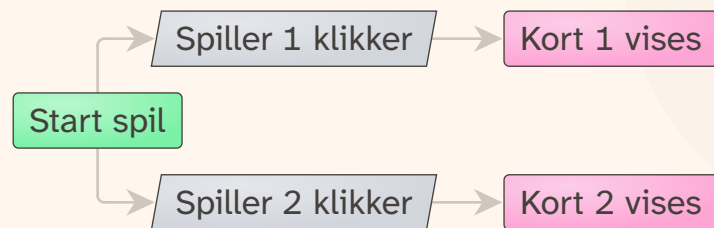
4.1 Et sted at starte: Ét spil kort

- 01 Start med en løsning hvor man kan klikke på (hvad der ligner) en stak kort for at trække det øverste fra stakken, som vises.
- 02 Læs API dokumentationen. Nederst på siden er der en URL til en billedetil af bagsiden af et kort, som I kan bruge.
- 03 De første to metoder kan bruges til at “initialisere” et nyt spil kort når siden loader, og til at trække et kort når man klikker på stakken. Det returnerede JSON respons indeholder en URL til et billede af kortet.

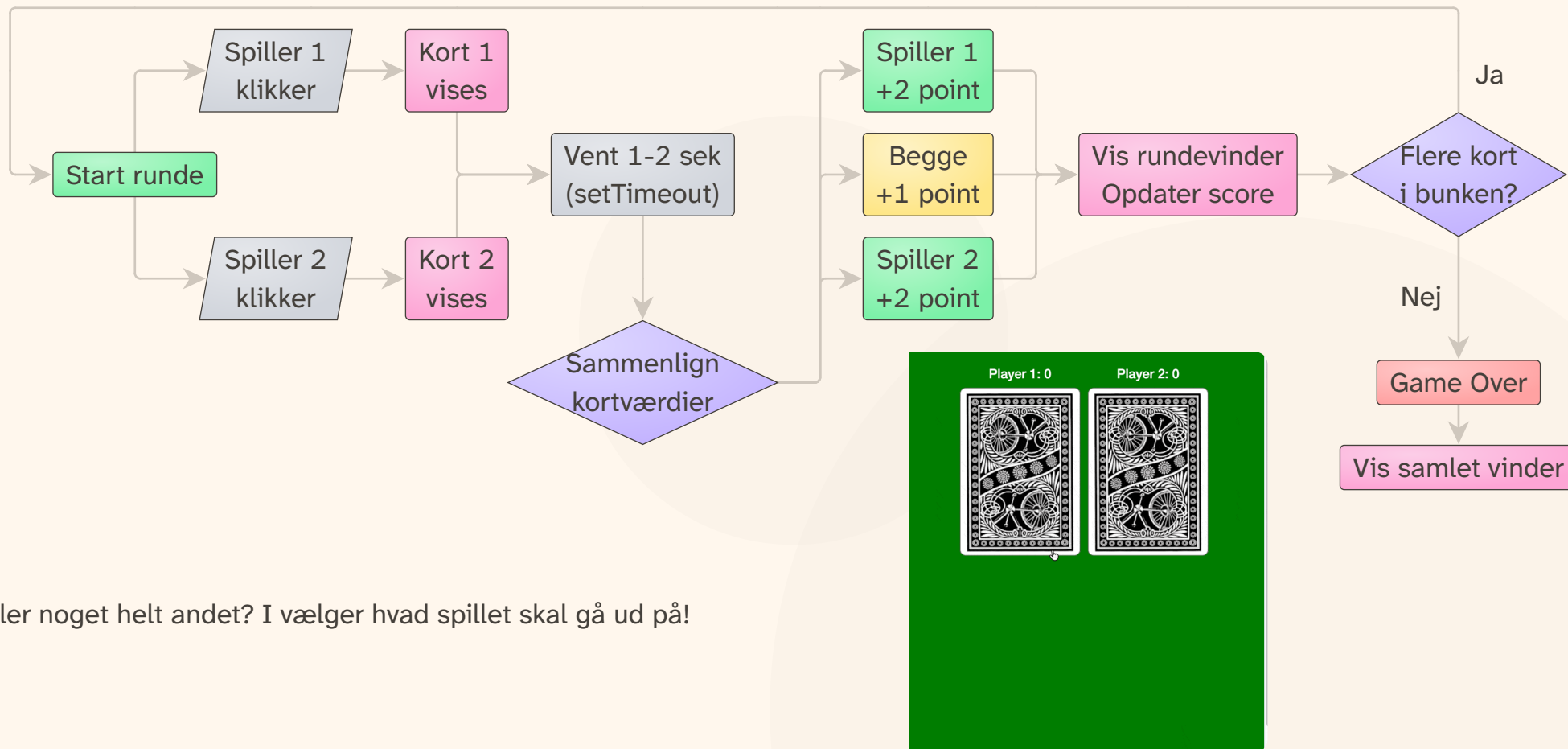


4.2 En mulig version 2: To spil kort

- 01 Start med en løsning hvor man kan klikke på (hvad der ligner) en stak kort for at trække det øverste fra stakken, som vises.
- 02 Her begynder der at være noget repetition, så overvej hvad I kan trække ud i genbrugelige funktioner.



4.3 En mulig version 3: Et spil krig



Eller noget helt andet? I vælger hvad spillet skal gå ud på!

4.4 Inspicér jeres kortspil i DevTools

🔗 Parvis ⌚ 10 min

🔍 Åbn Netværk-fanen og undersøg

Nu hvor I har bygget jeres kortspils-app, så lad os lige kigge på hvad der egentlig sker i browseren:

- 01 **Åbn DevTools** og gå til **Netværk**-fanen
- 02 **Sørg for at viewet er tomt** (tryk på clear/ryd-knappen)
- 03 **Interager med jeres app** — vend kort, træk nye kort, etc.
- 04 **Observer hvad der sker** i Netværk-fanen

💡 Hvad skal I lede efter?

- Hvilke **URL'er** bliver kaldt?
- Hvor **lang tid** tager hvert kald?
- Hvilken **HTTP-metode** bruges? (GET/POST/etc.)
- Hvor **store** er de responses, der kommer fra API'et?
- Kan I se **JSON-dataene** der kommer tilbage?

5. Vis jeres løsning ved bordet

👥 Grupper ⌚ 20 min

🔍 Find tilbage til jeres bord (I deles måske parvis)

Hver person (eller par, hvis I sidder ved samme bord) får **3 minutter** til at demonstrere deres løsning:

- **Vis** jeres kortspil i aktion
- **Forklar kort** jeres tilgang til at bruge Deck of Cards API'et
- **Fortæl** hvor I måtte slå op i dokumentation eller på anden måde søge hjælp
- **Highlight én udfordring** I stødte på og hvordan I løste den
- **Del én ting** I lærte om fetch/promises undervejs

Resten af bordet: Stil opklarende spørgsmål og notér interessante løsninger

6. Præsenter eksempler og læring for klassen

👥 Grupper ⌚ 20 min

💡 Hvert bord udvælger et eksempel til præsentation

Vælg det mest interessante eksempel ved bordet, som I vil dele med klassen. Måske den mest avancerede løsning, eller måske en interessant måde at strukturere koden, eller den største læring.

- Skaberne af den udvalgte løsningen **demonstrerer** den for klassen
- **Forklar kort** jeres tilgang til at bruge Deck of Cards API'et
- **Fortæl** hvor I måtte slå op i dokumentation eller på anden måde søge hjælp
- **Highlight én udfordring** I stødte på og hvordan I løste den
- **Del det mest interessante** I lærte om undervejs

Alle I andre: Stil opklarende spørgsmål og notér interessante løsninger